

算法设计与分析

第 10 章计算复杂性理论笔记

基于 PPT/PDF 的图文整理笔记

原始课件：第 10 章计算复杂性理论

整理者：InkTH

日期：2026 年 5 月 29 日

阅读说明

黑色正文表示来自课件定义、性质、证明思路与例题结构的忠实整理；浅蓝色提示框表示为理解当前知识点而加入的少量解释、提醒或直觉说明。

目录

1 计算复杂性的基本观点	2
1.1 算法效率与问题本质	2
1.2 问题规模与渐近时间复杂度	2
1.3 易解问题与难解问题	3
2 P、NP、NPC 与 NP-hard	3
2.1 判定问题与 P 类	3
2.2 NP 类与不确定性算法	3
2.3 NP 完全问题与 NP 难问题	4
3 多项式时间归约	5
3.1 归约的含义	5
3.2 归约的性质	5
3.3 证明 NPC 的一般方法	5
4 SAT、3-CNF-SAT 与归约起点	5
4.1 Circuit-SAT 与 SAT	5
4.2 SAT 属于 NP	6
4.3 从 Circuit-SAT 到 SAT	7
4.4 3-CNF-SAT	7
5 典型 NP 完全问题	8
5.1 团问题 Clique	8
5.2 3-CNF-SAT 到 Clique 的归约	8
5.3 顶点覆盖问题 Vertex-Cover	9
5.4 Clique 到 Vertex-Cover 的归约	10
5.5 哈密尔顿回路问题 Ham-Cycle	10
5.6 Vertex-Cover 到 Ham-Cycle 的归约	11
5.7 旅行商问题 TSP	12
5.8 子集和问题 Subset-Sum	13
6 本章要点	14

本节知识体系

本节知识体系

本章围绕如何判断一个问题在计算上是易解还是难解展开。主线不是单独分析某个算法，而是从判定问题、时间复杂度、多项式时间算法、不确定性验证、归约这些概念出发，对问题本身进行分类。

- 第一部分先说明计算复杂性的观察角度：当输入规模增大时，用渐近时间复杂度评估求解代价，并据此区分易解问题与难解问题。
- 第二部分给出 **P**、**NP**、**NPC**、**NP-hard** 四类问题的定义与相互关系，其中 **P** 关注多项式时间求解，**NP** 关注多项式时间验证，**NPC** 是 **NP** 中最难的一层。
- 第三部分以多项式时间归约为核心工具，说明为什么“把一个已知难问题归约到新问题”可以用来证明新问题至少同样难。
- 第四部分沿着一条典型归约链展开：**Circuit-SAT** $\rightarrow$ **SAT** $\rightarrow$ **3-CNF-SAT** $\rightarrow$ **Clique** $\rightarrow$ **Vertex-Cover** $\rightarrow$ **Ham-Cycle** $\rightarrow$ **TSP**，并补充 **3-CNF-SAT** $\rightarrow$ **Subset-Sum**。

1 计算复杂性的基本观点

1.1 算法效率与问题本质

计算机性能能否真正发挥出来，取决于求解问题所采用的算法。分析一个问题时，不仅要设计数据结构和计算步骤，还要进一步区分哪些问题本身容易求解，哪些问题即使看上去相似却在本质上更难。

一些典型对照包括：

- 哈密尔顿回路问题与欧拉回路问题；
- 单源最长路径问题与单源最短路径问题；
- 3-CNF 可满足性问题与 2-CNF 可满足性问题。

Explanation

这类对照说明，表面形式相近并不意味着计算难度相近。复杂性理论关心的正是这种“结构上只差一点，但求解代价完全不同”的现象。

1.2 问题规模与渐近时间复杂度

问题的计算复杂性是问题规模的函数，因此首先需要定义输入规模。算法是求解问题的一系列计算步骤，而算法分析主要考察两类资源：

- **时间复杂度**：运行步骤数随输入规模增长的趋势；
- **空间复杂度**：所需存储空间随输入规模增长的趋势。

实际分析中通常不要求精确运行时间，而更关注输入很大时的增长趋势。若

$$f(n) = n^2 \log n + 10n^2 + n,$$

则其增长主导项是 $n^2 \log n$ ，因此运行时间表现为基于 $n^2 \log n$ 的渐近增长。

1.3 易解问题与难解问题

按照求解算法的时间复杂度，可以把问题粗略分成三类：

1. 存在多项式时间算法的问题；
2. 可以证明不存在多项式时间算法的问题；
3. 尚未找到多项式时间算法，也尚不能证明其不存在的问题。

通常把**存在多项式时间算法**的问题看作易解问题，把**需要超多项式时间**才能解决的问题看作难解问题。多项式时间算法的典型形式为

$$O(n^c), \quad c \text{ 为常数.}$$

像 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 、 $O(n^{100})$ 都属于多项式时间；而 $O(2^n)$ 、 $O(n!)$ 不属于多项式时间。

2 P、NP、NPC 与 NP-hard

2.1 判定问题与 P 类

本章讨论的核心对象是**判定问题**，即答案只有 yes/no 的问题。

若一个判定问题能用**确定性算法**在多项式时间内求解，则它属于 **P 类**。确定性算法是指：在每一步执行中都只有一个确定选择，因此对同一输入反复运行，输出始终相同。

P 类问题的定义可以表述为：存在一个确定性算法，在输入规模为 n 时，能在 $O(n^k)$ 步内给出 yes/no 答案，其中 k 是某个非负整数。

常见例子包括排序问题、搜索问题等。

2.2 NP 类与不确定性算法

若一个判定问题的某个候选解能够在**多项式时间内被验证正确**，则该问题属于 **NP 类**。

从不确定性算法的角度看，求解过程包含两个阶段：

- **guessing phase**：猜测一个候选解；
- **verification phase**：验证该候选解是否正确。

因此，NP 可以理解为“一类可以在多项式时间内验证解”的判定问题，并且

$$P \subseteq NP.$$

旅行商问题的判定形式是一个典型例子：虽然目前没有已知的多项式时间确定性算法来找到最小周游路线，但若给出一条候选路线，就可以在多项式时间内检查：

- 是否经过每个城市一次且仅一次；
- 是否形成合法回路；
- 总长度是否不超过给定上界 L 。

Tips

“属于 NP” 并不等于 “已经知道怎么高效求解”。它只说明：如果有人把答案交出来，就能较快验收。

2.3 NP 完全问题与 NP 难问题

一个问题 D 是 **NP 完全问题**，需要同时满足：

1. $D \in NP$ ；
2. NP 中任何问题都能在多项式时间内归约为 D 。

因此，NP 完全问题是 **NP 类中最难的一层**。如果其中任意一个问题能被多项式时间确定性算法求解，那么 NP 中所有问题都能在多项式时间内求解，也即 $P = NP$ 。

若一个问题满足上面的第二条，但不一定满足第一条，则称为 **NP 难问题**。NP 难问题不一定属于 NP，例如图灵停机问题。它至少与 NP 完全问题一样难，甚至可能更难。

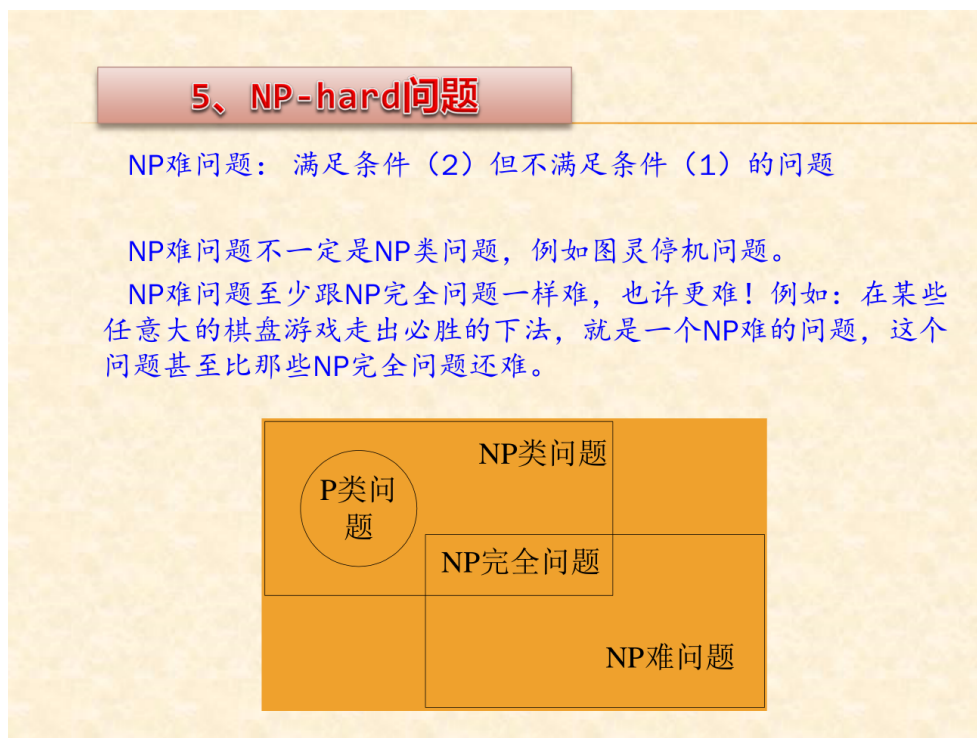


图 1 P、NP、NP 完全与 NP 难问题之间的包含关系

3 多项式时间归约

3.1 归约的含义

为了判断问题的难易程度，可以通过一个问题与另一个问题之间的转换关系进行比较。

若问题 A 的任意实例 α 都能转换成已知问题 B 的某个实例 β ，并且：

- 转换过程只需多项式时间；
- β 的解能够对应回 α 的解；

则说明 A 可以借助 B 的解法求解，这种关系称为**多项式时间归约**，记作

$$A \leq_p B.$$

形式化地说，若 L_1 与 L_2 分别表示两个问题， $L_1 \leq_p L_2$ 表示存在一个多项式时间函数 f ，使得 L_1 的任意实例 x 都能被映射成 $f(x)$ ，并满足归约前后的 yes/no 输出一致。

3.2 归约的性质

多项式时间归约具有以下性质：

- **传递性**：若 $A \leq_p B$ 且 $B \leq_p C$ ，则 $A \leq_p C$ ；
- **易解性传递**：若 $A \leq_p B$ 且 $B \in P$ ，则 $A \in P$ ；
- **难解性传递**：若 $A \leq_p B$ 且 A 是难问题，则 B 至少同样难。

归约提供了一种形式化方法，用来证明一个问题在多项式时间因子内至少与另一个问题一样难。若已知 L_1 是 NPC 问题，且 $L_1 \leq_p L_2$ ，再加上 $L_2 \in NP$ ，就可推出 L_2 也是 NPC 问题。

3.3 证明 NPC 的一般方法

证明一个问题 L 是 NP 完全问题，常用两步法：

1. 证明 $L \in NP$ ；
2. 选择一个已知的 NP 完全问题 L' ，证明 $L' \leq_p L$ 。

直接依据定义去证明“任意 $L' \in NP$ 都能归约到 L ”通常过于困难，因此实际中更常采用第二种方法。

4 SAT、3-CNF-SAT 与归约起点

4.1 Circuit-SAT 与 SAT

布尔组合电路的可满足性问题是第一个被证明为 NP 完全的问题。问题形式为：**给定一个逻辑电路，判断是否存在一种输入使输出为 True。**

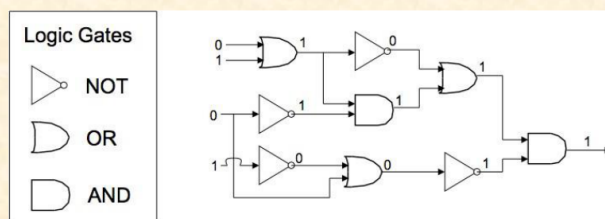
第一个NPC问题

电路的可满足性问题：

给定一个逻辑电路，判断该逻辑电路是否存在一种输入，使输出为True。

其它的NPC问题都是由这个问题归约而来。因此，逻辑电路问题是NPC类问题的“鼻祖”。

再证明一个新的问题是NPC问题，只需要将一个已知的NPC问题归约为该新问题即可。



布尔组合电路的可满足性(忽略硬件技术环节)

图 2 电路可满足性问题作为后续归约链的起点

布尔表达式可满足性问题 SAT 的一个实例是布尔公式 P 。该公式由以下部分构成：

- n 个布尔变量 $x_1, x_2, \dots, x_n$ ；
- 若干布尔联结词，如合取、析取、否定、蕴含、等值；
- 括号。

一个真值指派是对所有变量赋 0/1 值；若某个指派能使公式为真，则称该公式可满足。

4.2 SAT 属于 NP

证明 SAT 属于 NP，只需说明：若给出一个可满足指派作为证书，则可在多项式时间内验证。

验证过程为：

1. 用给定布尔值替换公式中的每个变量；
2. 计算所得表达式的值；
3. 若结果为 1，则公式可满足。

该过程对输入规模 $n + m$ 来说是多项式时间的，因此 $SAT \in NP$ 。

4.3 从 Circuit-SAT 到 SAT

每个布尔电路都可以改写为布尔公式，因此可建立

$$\text{Circuit-SAT} \leq_p \text{SAT}.$$

直接把电路机械展开成公式可能因扇出导致规模膨胀，因此需要通过合适的构造保持转换规模为多项式级别。这样，电路的可满足性与转换后公式的可满足性保持一致。

4.4 3-CNF-SAT

一个**文字**是变量或变量的非；**合取范式**是若干子句的合取，每个子句是若干文字的析取；**3-CNF** 指每个子句恰好含有 3 个文字。

3-CNF 可满足性问题是：给定一个 3-CNF 形式的布尔公式，判断它是否可满足。

这个问题属于 NP，并且有

$$\text{SAT} \leq_p \text{3-CNF-SAT}.$$

转换思路包括三步：

1. 将一般布尔公式转为子句的合取式；
2. 将子句进一步整理成合取范式；
3. 把每个子句规范成含 3 个文字的形式。

上述每一步都只引入多项式规模增长，因此整体构成多项式时间归约。



图 3 典型 NP 完全问题之间的一条归约链

5 典型 NP 完全问题

5.1 团问题 Clique

无向图 $G = (V, E)$ 中的团是顶点子集 $V' \subseteq V$ ，其中任意两个顶点之间都有边相连，也即该子图是完全图。团的规模是所含顶点数。

团问题的判定形式是：图中是否存在一个规模为 k 的团。记作

$$\text{Clique} = \{ \langle G, k \rangle : G \text{ 含有规模为 } k \text{ 的团} \}.$$

这个问题属于 NP。证书可取为顶点集合 V' ，验证时只需检查：

- $|V'| = k$;
- 对任意 $u, v \in V'$ ，边 $(u, v) \in E$ 。

5.2 3-CNF-SAT 到 Clique 的归约

设

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$$

是一个 3-CNF 公式，每个子句含 3 个文字。构造归约图时：

- 每个子句对应一组顶点；

- 每个文字对应一个顶点；
- 若两个顶点来自不同子句，且对应文字不互为否定，则在它们之间连边。

于是可以得到：

- 若 φ 可满足，则每个子句可选出一个取值为真的文字，这些顶点两两相容，从而形成规模为 k 的团；
- 若图中存在规模为 k 的团，则这 k 个顶点分别来自不同子句，且彼此不矛盾，可据此构造一组可满足赋值。

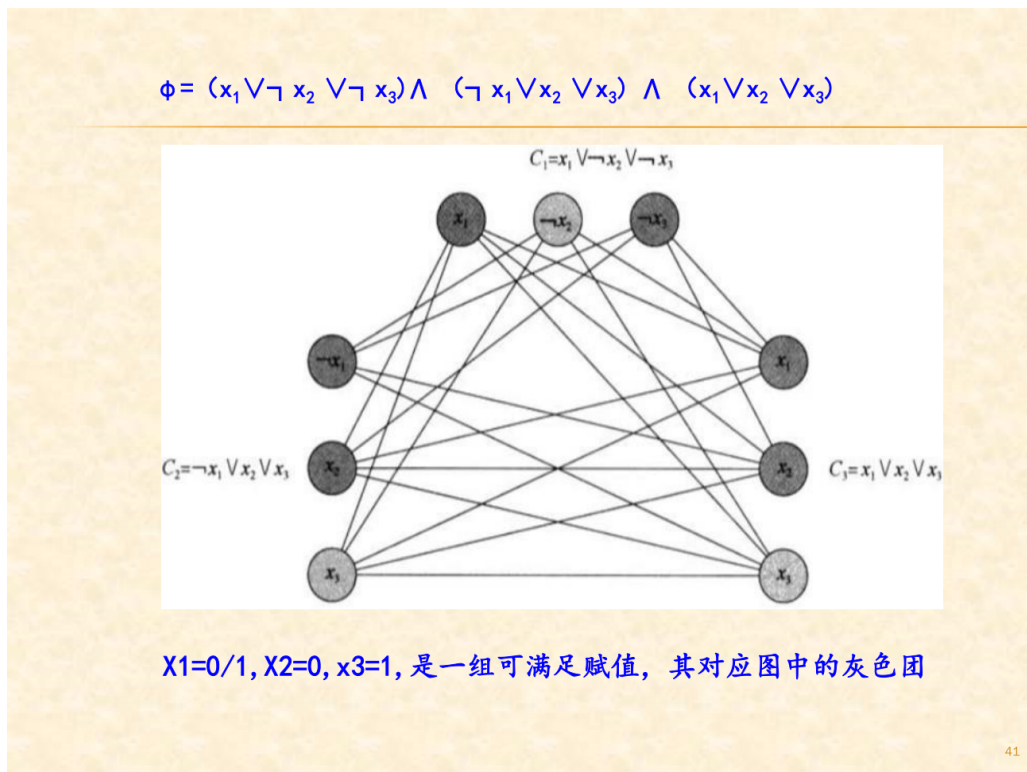


图 4 由 3-CNF 公式构造 Clique 实例的示意

Explanation

这个归约的关键不是“把公式翻译成图”，而是让从每个子句中选一个彼此相容的文字这件事，精确对应为在图中选出一个团。

5.3 顶点覆盖问题 Vertex-Cover

顶点覆盖问题要求在图中寻找最小规模的顶点覆盖。其判定形式是：给定图 G 与整数 k ，图中是否存在规模为 k 的顶点覆盖。记作

$$\text{Vertex_Cover} = \{ \langle G, k \rangle : G \text{ 有规模为 } k \text{ 的顶点覆盖} \}.$$

它也属于 NP。证书可取为顶点集合 $V' \subseteq V$ ，验证时检查 $|V'| = k$ ，并对每条边 $(u, v) \in E$ 判断是否有 $u \in V'$ 或 $v \in V'$ 。

5.4 Clique 到 Vertex-Cover 的归约

设 $G = (V, E)$, 其补图记作 $\bar{G} = (V, \bar{E})$, 其中

$$\bar{E} = \{(u, v) : u, v \in V, u \neq v, (u, v) \notin E\}.$$

则有:

$$Clique \leq_p Vertex_Cover.$$

归约关系为:

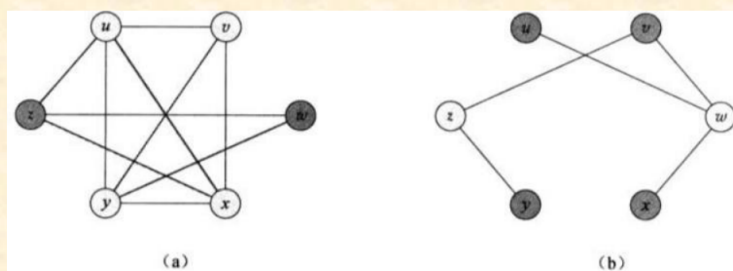
G 中有规模为 k 的团 $\iff \bar{G}$ 中有规模为 $|V| - k$ 的顶点覆盖.

■ 团问题可以归约为顶点覆盖问题

$$Clique \leq_p Vertex_Coer$$

证明: 已知一个无向图 $G = (V, E)$ 定义 G 的补图 $\bar{G} = (V, \bar{E})$,

其中 $\bar{E} = \{(u, v) : u, v \in V, u \neq v, (u, v) \notin E\}$



归约过程: 图 G 具有一个规模为 k 的团, 当且仅当图 $\bar{G}$ 有一个规模为 $|V| - k$ 的顶点覆盖

图 5 团与补图中的顶点覆盖之间的对应关系

5.5 哈密顿回路问题 Ham-Cycle

哈密顿回路是经过图中每个顶点一次且仅一次, 并最终回到起点的闭合路径。判定问题是: 给定一个图, 判断它是否包含哈密顿回路。

这个问题属于 NP。证书是一个顶点序列, 验证时检查:

- 是否恰好包含每个顶点一次;
- 相邻顶点之间以及首尾之间是否都有边。

5.6 Vertex-Cover 到 Ham-Cycle 的归约

顶点覆盖到哈密顿回路的归约通过构造图 G' 完成。对原图 G 的每条边，都引入一个**附件图 (widget)**；附件图共有 12 个顶点和 14 条边，并且顶点的遍历路径只允许两种模式：

- 穿过全部节点；
- 只穿过一侧节点。

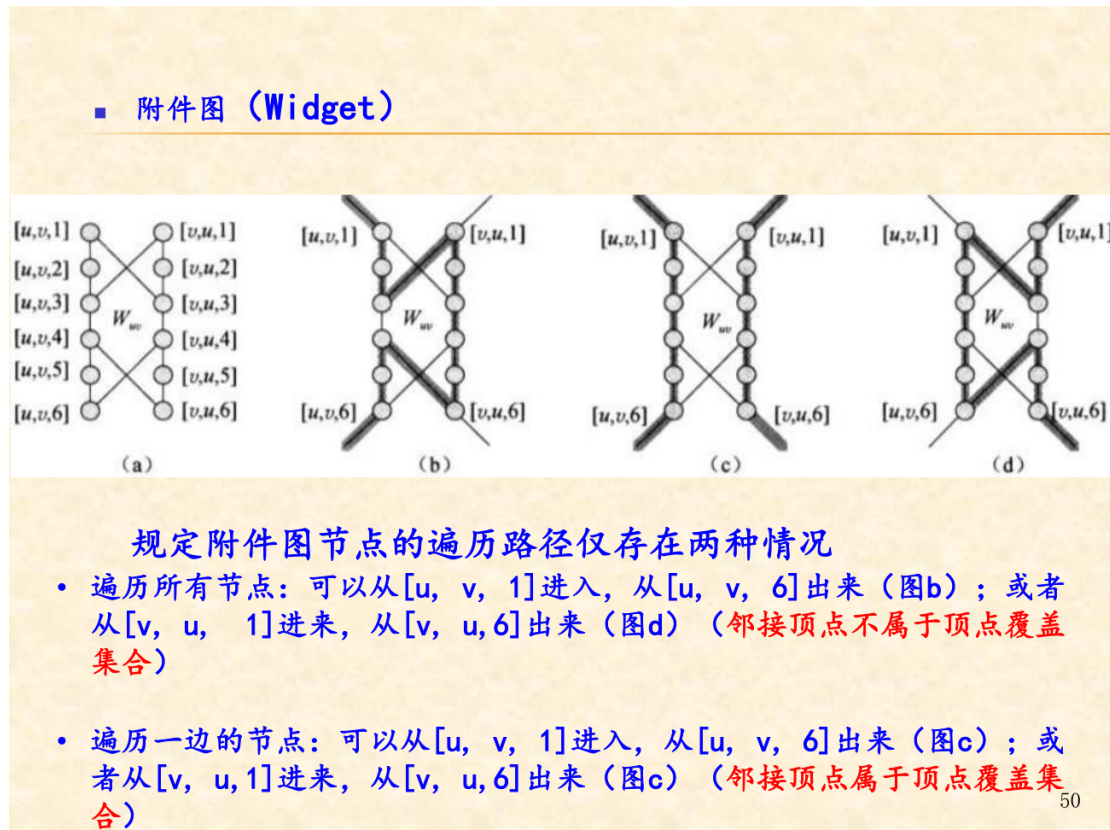


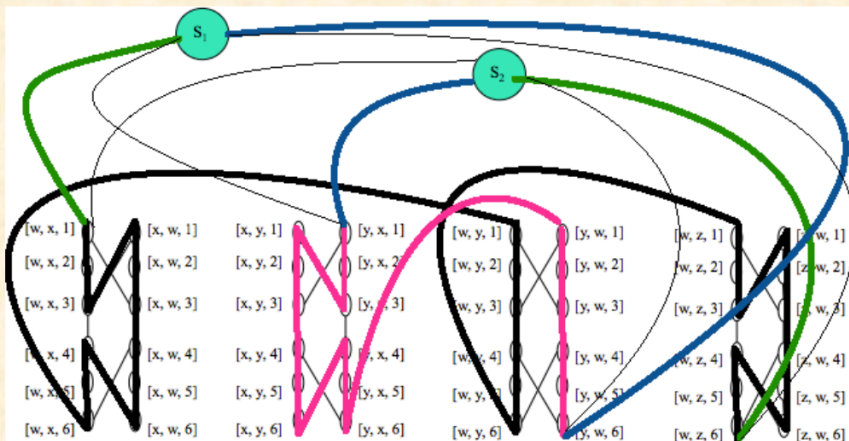
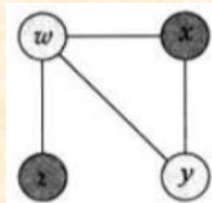
图 6 Vertex-Cover 到 Ham-Cycle 归约中的附件图结构

随后再把多个附件图与选择器顶点连接起来，构造出最终的 G' 。这样：

- 若 G 存在规模为 k 的顶点覆盖，则 G' 存在哈密顿回路；
- 若 G' 存在哈密顿回路，则可从选择器对应关系恢复出 G 的一个顶点覆盖。

■ 图 G'

- 连接选择器顶点和附件图的边: 将每个顶点所对应的附件图中的第一个顶点和以及与其相邻接的排列中的最后一个顶点与每个选择器顶点相连
- 加粗的为最终的哈密顿回路



56

图 7 最终构造出的 G' 及其哈密顿回路示意

Note

这一归约的核心是把“某个顶点是否进入覆盖集”编码成“回路如何穿过对应附件图”。构造较复杂，但逻辑仍然是让原问题中的合法选择与新问题中的合法回路一一对应。

5.7 旅行商问题 TSP

旅行商问题可用一个有 n 个顶点的完全图建模。每条边 (i, j) 带有成本 $C(i, j)$ ，目标是寻找一个成本最小的哈密顿回路。

其判定形式为：是否存在一条总成本不超过给定上界的周游回路。这个问题属于 NP，因为给定一个候选回路后，可以在多项式时间内检查顶点是否恰好访问一次，并累加边权验证总成本。



图 8 旅行商问题的图模型与一条最低费用回路示例

从哈密尔顿回路问题到旅行商问题的归约为：

$$Ham_Cycle \leq_p TSP.$$

构造方法是：对 $G = (V, E)$ 扩充得到完全图 G' ，并定义成本函数

$$C(i, j) = \begin{cases} 0, & (i, j) \in E, \\ 1, & (i, j) \notin E. \end{cases}$$

于是，若 G 中存在哈密尔顿回路，就能在 G' 中找到总成本为 0 的旅行商回路；反之，若 G' 中存在满足阈值的回路，则该回路对应于 G 中的哈密尔顿回路。

5.8 子集和问题 Subset-Sum

子集和问题的判定形式为：

$$Subset_Sum = \{ \langle S, t \rangle : \exists S' \subseteq S, t = \sum_{A \in S'} A \}.$$

即给定一个有限集合 $S \subset \mathbb{N}$ 和目标值 $t \in \mathbb{N}$ ，判断是否存在子集 S' 使得元素和恰好为 t 。

子集和问题

给定一个有限集合 $S \subset N$, 以及一个目标值 $t \in N$ 。是否有 S 的子集 S' 使得 $t = \sum_{A \in S'} A$

例如: $S = \{1, 3, 5, 6, 7, 9, 10\}$

$t = 19$

Subset_Sum = $\{ \langle S, t \rangle : \text{存在一个子集 } S' \subseteq S, \text{ 使得 } t = \sum_{A \in S'} A \}$

62

图 9 子集和问题的定义与数值示例

它属于 NP。证书可取为候选子集 S' ，验证时只需把 S' 中元素求和，并检查结果是否等于 t 。

此外，还有归约关系：

$$3\text{-CNF-SAT} \leq_p \text{Subset_Sum}.$$

该构造通过把变量选择与子句满足关系编码到数字位上，使“存在满足赋值”与“存在和为目标值的子集”对应起来，因此子集和问题也是 NP 完全问题。

6 本章要点

Summary

- 计算复杂性研究的是**问题**随规模增长时的求解代价，而不是某次具体运行的机器时间。
- P 类问题能在**确定性多项式时间**内求解；NP 类问题能在**多项式时间**内验证候选解。
- NP 完全问题同时满足“属于 NP”与“NP 中任意问题都能归约到它”，因此它们是 NP 中最难的一层。
- 证明新问题是 NPC，通常先证 $L \in NP$ ，再从一个已知 NPC 问题出发证明 $L' \leq_p L$ 。

- Circuit-SAT、SAT、3-CNF-SAT、Clique、Vertex-Cover、Ham-Cycle、TSP、Subset-Sum 构成了本章最重要的一组典型问题与归约关系。